

Istruzioni condizionali

Today

Istruzioni condizionali in Python

Concetto base

- **Istruzione condizionale:** permette di eseguire blocchi di codice solo se una **condizione** risulta vera; altrimenti si può eseguire una alternativa. Esempi quotidiani: *se piove prendo l'ombrello, se la spia della benzina è bassa faccio rifornimento.*

Sintassi e indentazione

Elemento	Descrizione
if	Parola chiave che introduce la condizione (non può essere usata come nome di variabile per evitare confusione).
condizione	Espressione che viene valutata come True o False.
:	Due punti obbligatori dopo la condizione.
blocco indentato	Una o più istruzioni tutte con la stessa spaziatura verso destra (di solito Tab).
else (opzionale)	Blocca l'alternativa da eseguire se la condizione è False.
elif (opzionale)	Contrazione di else+if per gestire più condizioni in cascata (vedi sotto).

- L'**indentazione** è significativa: indica a Python quali istruzioni appartengono a ciascun blocco.
- La sintassi può essere corretta ma il programma comunque non funzionare correttamente se la logica o i tipi sono sbagliati (errore semantico).

Visualizzazione e semantica: flowchart

- Le conditional si visualizzano spesso con un **flow chart**: una decisione (condizione) con due uscite: **True** (esegue un blocco) e **False** (esegue l'altro blocco o salta).
- Nel caso di più controlli su **una stessa variabile** e intervalli disgiunti, si crea una **cascata**: si verifica la prima condizione, se è False si passa alla successiva, e così via fino a trovare un True o terminare.

Esempio pratico: classificare un voto (esercizio)

Testo dell'esercizio: chiedere in input un intero tra 0 e 10 e stampare una sola etichetta corrispondente:

Intervallo	Output
$V < 6$	insufficiente
$6 \leq V < 8$	sufficiente
$8 \leq V < 10$	ottimo
$V = 10$	eccellente

Note:

- L'esercizio richiede che venga stampata **solo** l'etichetta corrispondente al corretto intervallo (se $V=10$ deve stampare soltanto "eccellente").
- Si può implementare usando if separati, if-else nidificati o la sequenza if / elif / else (consigliata per chiarezza).

Input da utente e tipi: errore comune ⚠️

```
"""<' not supported between instances of 'str' and 'int'"""
```

Risultato: quando si usa `input()` in Python il valore acquisito è di tipo **str** (stringa). Confronti numerici con interi causano `TypeError`.

Soluzione: convertire l'input al tipo numerico desiderato, ad esempio:

- usare `int(input(...))` se ci si aspetta un intero;
- eventualmente controllare e gestire errori con `try/except` per input non numerici.

Esempio corretto:

- `voto = int(input("Che voto hai preso? "))`

Controlli a cascata: comportamento e quando usarli 🌊

- **Cascata:** serie di controlli uno dopo l'altro sulla stessa variabile, con intervalli mutualmente esclusivi. Appena si trova una condizione `True`, le successive non vengono più testate (si "salta" il resto).
- Condizione valida per la cascata:
 - si controlla **la stessa variabile**;
 - i range/condizioni sono **disgiunti** (non sovrapposti), quindi una sola può essere vera alla volta.
- Se i range si sovrappongono, la logica a cascata potrebbe produrre risultati inattesi e non è appropriata.

Modi equivalenti di scrivere la stessa logica

- Serie di `if` separati (tutti sullo stesso livello): ogni `if` viene valutato indipendentemente — può essere utile se più condizioni possono essere vere contemporaneamente.

- `if / else` nidificati: producono un effetto "scalinato" (ogni `else` contiene un altro `if`), ma tendono a spostare il codice sempre più a destra con molti livelli.
- `if / elif / else`: forma più pulita per gestire più controlli esclusivi sulla stessa variabile (nessun ulteriore indent e chiaro effetto a cascata).

Buone pratiche e suggerimenti

- Convertire subito l'input al tipo corretto (es. `int(input(...))`) per evitare `TypeError`.
- Preferire `if / elif / else` per catene di condizioni mutualmente esclusive sulla stessa variabile: è leggibile e previene nidificazioni eccessive.
- Visualizzare la logica con un diagramma (flowchart) se la sequenza è complessa: aiuta a capire quali rami sono mutuamente esclusivi e quale sarà il punto di convergenza finale.
- Testare il programma con valori ai limiti (es. 5, 6, 7.999, 8, 9, 10) per verificare gli intervalli e l'esclusività delle condizioni.

Controlli condizionali annidati e il problema dell'indentazione

- **Problema:** annidare molti `if / else` uno dentro l'altro causa un progressivo spostamento a destra del codice (indentazione crescente), diminuendo leggibilità e aumentando rischio di errori.
- **Sintomo pratico:** per ogni livello annidato l'indentazione aumenta; con decine di controlli il codice "si sposta" molto verso destra e diventa difficile da leggere e mantenere.

Soluzione 1 – `elif`

- `Elif` è la contrazione di `else if` e permette di scrivere catene di controlli a cascata senza annidare profondamente.

- **Effetti:**

- Testa ogni condizione solo se la precedente è falsa (comportamento a cascata).
- Riduce l'indentazione e rende il codice più verticale e leggibile.
- Codice più compatto e meno soggetto a spostamenti verso destra.

- **Quando usarlo:** quando si vuole una serie di condizioni mutualmente esclusive legate alla stessa variabile/contesto (es.: classificazione di voti).

- **Esempio:**

```
if voto < 6:  
    print("Insufficiente")  
elif 6 <= voto < 8:  
    print("Sufficiente")  
elif 8 <= voto < 10:  
    print("Ottimo")  
else:  
    print("Eccellente")
```

Match-case (switch-like) in Python

- **Scopo:** confrontare una singola variabile con un insieme di valori discreti in modo chiaro e a prova di errori di sovrapposizione.
- **Sintassi base:**


```
match voto:
    case 0 | 1 | 2 | 3 | 4 | 5:
        print("Insufficiente")
    case 6 | 7:
        print("Sufficiente")
    case 8 | 9:
        print("Ottimo")
    case 10:
        print("Eccellente")
    case _:
        print("Voto non valido")
```

- **Caratteristiche importanti:**

- Usa l'operatore `|` per specificare alternative (es.: `case 6 | 7`).
- `case _` è il caso di default (qualsiasi altro valore).
- Ideale per valori discreti e mutuamente esclusivi; meno adatto per intervalli o condizioni con operatori `<`, `>` direttamente (per questi è meglio `if / elif`).
- Supporta pattern matching strutturale (es.: spaccettare liste/tuple), ma richiede attenzione alla sintassi.

- **Vantaggi:**

- Riduce errori di battitura che portano a condizioni sovrapposte.
- Garantisce che ogni valore sia trattato in modo esclusivo (quando il problema è di matching su un insieme finito di valori).

- **Nota pratica:** imparare prima `if / elif` è consigliato; `match` è utile in casi specifici e non sempre necessario.

Indentazione in Python: significato e impatto sul flusso ☁️

- **Principio fondamentale:** in Python la spaziatura (indentazione) delimita i blocchi di istruzioni. Non ci sono parentesi graffe: è la posizione orizzontale che determina l'appartenenza a un blocco.

- **Regole:**

- Tutte le istruzioni di un blocco devono essere indentate dello stesso numero di spazi o tab (non mischiare spazi e tab).
- Almeno un livello di indentazione è richiesto dopo `if` , `for` , `while` , `def` , ecc.
- Se manca l'indentazione attesa si ottiene un errore di tipo `IndentationError: expected an indented block` .

- **Esempio pratico (flusso diverso a seconda dell'indentazione):**

- Codice A (chiede sempre se piove):

```
temperatura = float(input("Quanti gradi ci sono? "))
if temperatura < 28:
    print("Prendi la giacca")
piove = input("Sta piovendo? ")
if piove == "sì":
    print("Prendi l'ombrello")
print("Buon giro!")
```

- Codice B (chiede se piove SOLO quando temperatura < 28):

```
temperatura = float(input("Quanti gradi ci sono? "))
if temperatura < 28:
    print("Prendi la giacca")
    piove = input("Sta piovendo? ")
    if piove == "sì":
        print("Prendi l'ombrello")
print("Buon giro!")
```

- **Conseguenze:**

- Nel Codice A l'input sulla pioggia viene richiesto sempre, indipendentemente dalla temperatura.
- Nel Codice B l'input viene richiesto solo se la temperatura è inferiore a 28, perché è indentato all'interno del `if` .
- Spostando ulteriormente a destra altre istruzioni (più livelli di indentazione) si ottengono comportamenti ancora più

condizionati (es.: "Buon giro!" eseguito sempre vs. solo in alcuni casi).

- **Implicazione pratica:** cambiando di poco l'indentazione si possono ottenere programmi con comportamenti completamente diversi – attenzione quando si inseriscono nuove istruzioni dentro o fuori un blocco.

Tabella riassuntiva: costrutti e uso

Costrutto	Quando usarlo	Vantaggi
if	Singola condizione o condizioni non mutualmente esclusive	Semplice, flessibile (consente disuguaglianze)
elif	Catena di condizioni mutualmente esclusive	Riduce annidamento, migliora leggibilità
else	Caso finale quando tutte le precedenti sono false	Copre l'ultimo caso
match-case	Variabile che può assumere un insieme discreto di valori	Previene sovrapposizioni, sintassi chiara per valori enumerati
Indentazione	Delimita blocchi di codice in Python	Necessaria per il corretto flusso; attenzione alla posizione delle istruzioni

Consigli pratici per lo studio e la scrittura del codice

- Preferire `elif` quando si hanno più condizioni alternative per la stessa variabile per evitare annidamenti profondi.
- Usare `match-case` quando si deve confrontare una variabile con un

insieme finito di valori distinti (e usare `case _` per il default).

- Controllare sempre l'indentazione: visualizzare i livelli e testare i casi limite (es.: valore minimo, massimo, non valido).
- Fare piccoli esempi e provare tutte le combinazioni di input per capire il comportamento del programma (specialmente con spostamenti di indentazione).
- Se si ha una logica basata su intervalli (es.: `voto < 6` , `6 <= voto < 8`), `if / elif` rimane la scelta più naturale.

Calcolo del rischio cardiovascolare

- **Problema descritto:** si vuole calcolare un coefficiente di rischio cardiovascolare in base alla **temperatura** e al fatto che **piova** o meno. L'autore ha scritto tre assegnazioni consecutive alla stessa variabile (una per ciascun caso), ma nessuna struttura condizionale adeguata: la variabile viene sovrascritta più volte e il valore finale stampato è sempre l'ultimo assegnato, indipendentemente dalla temperatura o dalla pioggia.
- **Comportamento errato:** scrivere tre righe come
 - `rischio = ...`
 - `rischio = ...`
 - `rischio = ...` porta a ottenere sempre il valore dell'ultima riga, quindi il programma non differenzia i casi.
- **Correzione logica:** usare una struttura **condizionale** (ad es. `if / elif / else`) per assegnare **una sola volta** il valore corretto di rischio in base al caso, oppure usare variabili diverse per ogni situazione (più leggibile ma meno idiomático).

| "prendi la giacca"

Questa è l'istruzione che il programma dovrebbe stampare quando la temperatura è bassa; va seguita da una domanda sul meteo (es. se sta piovendo).

Questa è l'istruzione che il programma dovrebbe stampare quando, a temperatura bassa o comunque previsto, risulta che **sta piovendo**; è parte della logica di output condizionale.

"calcola il rischio cardiovascolare moltiplicando la temperatura per 0,005"

Qui viene definita la formula proposta per il caso di pioggia; matematicamente si intende moltiplicare la temperatura per 0.005 per ottenere il coefficiente di rischio.

Errori comuni e buone pratiche

- **Sovrascrittura di variabili:** evitare di assegnare ripetutamente la stessa variabile senza separare i casi con if/elif/else; l'ultima assegnazione annulla le precedenti.
- **Uso corretto di if/elif/else:** strutturare i casi in modo esclusivo, ad esempio:
 - if temperatura < 28: ...
 - elif temperatura >= 28: ...
 - all'interno del primo ramo, nidificare la verifica della pioggia.
- **Leggibilità:** preferire nomi espliciti (es. **rischio_pioggia**, **rischio_asciutto**) o, meglio, assegnare una sola volta **rischio** all'interno di blocchi condizionali ben formati.
- **Test dei casi:** provare più input (es. 15°C con pioggia, 15°C senza pioggia, 30°C) per verificare che l'output sia coerente.

Interazione con modelli (es. ChatGPT)

- Un modello intelligente può rilevare la sovrascrittura e suggerire la struttura condizionale corretta: ad esempio, notare che il valore è calcolato tre volte ma vengono considerati solo gli ultimi risultati.

- Le correzioni proposte dai modelli vanno comunque verificate con test pratici e casi limite.

Condizionali vs Iterazioni

- **Condizionale (if):**
 - Testa una condizione e **sceglie** quale ramo eseguire (di solito due o più rami).
 - Ha frecce d'uscita (es. true / false) ma non ritorna al punto di controllo.
- **Iterazione / loop:**
 - Ripete un blocco di istruzioni più volte fintanto che una condizione è vera o per un numero prestabilito di volte.
 - Nel diagramma di flusso ha una freccia che torna alla condizione (back edge), creando un ciclo.
 - Si usa quando occorre applicare la stessa operazione a molti elementi (es. un milione di numeri).

Aspetto	Condizionale	Iterazione (loop)
Scopo	Scelta di percorso	Ripetizione di istruzioni
Uscite dal blocco	Non ritorna al controllo	Spesso ritorna al controllo (loop)
Quando usarlo	Diversi comportamenti in base a condizioni	Applicare stesso controllo/operazione su molti elementi

Perché i loop sono essenziali – esempi pratici



- Esempio: contare quanti numeri pari in un milione di valori. Scrivere un milione di if sarebbe impossibile; invece si usa un loop che scorre i numeri e applica una singola condizione a ciascuno.

- Applicazioni moderne: l'**intelligenza artificiale** e i modelli numerici eseguono ripetute operazioni su miliardi di parametri – tutto si basa su iterazioni efficienti.

Tipi di dati principali (ricapitolazione) 🧱

Tipo	Descrizione
int	Numeri interi (es. 1, -5, 0)
float	Numeri con virgola (es. 0.005, 3.14)
bool	Valori booleani: True / False
str	Catene di caratteri (testo)

Note pratiche per l'esame e sviluppo 🖋️

- Scrivere codice leggibile e testabile: nomi chiari, commenti essenziali, casi di test.
- Piccole varianti sintattiche (es. usare operatori composti come `/=` o `*=`) sono accettabili se il risultato è corretto e leggibile.
- Prossimi argomenti da studiare: **iterazioni (loop)**, **funzioni** – servono per risolvere esercizi più complessi in modo modulare.